

TP2 - Relatório de Refatoração

1. Análise do Código Original

Ao analisar o código, identifiquei os seguintes problemas:

1.1 Pipeline.java

- **Método longo:** O método `run()` tinha aproximadamente 46 linhas fazendo múltiplas coisas
- **Strings mágicas:** Valores como "success", "Tests failed", "Deployment completed successfully" espalhados pelo código
- **Duplicação de lógica:** `"success".equals(...)` aparecia múltiplas vezes
- **Condicionais aninhados:** 3 níveis de if-else para determinar a mensagem de email
- **Baixa coesão:** Uma única classe fazendo testes, deploy e envio de email

1.2 PipelineTest.java

- Apenas um método placeholder `todo()` sem implementação real
- Zero cobertura de testes

2. Decisões de Refatoração

2.1 Priorização dos Testes

Decidi começar escrevendo os testes primeiro, porque:

- Os testes funcionam como rede de segurança
- Qualquer mudança de comportamento seria detectada imediatamente
- Facilita a compreensão do comportamento esperado

Criei 8 testes cobrindo:

- Sucesso completo (testes passam + deploy ok)
- Falha nos testes
- Falha no deploy
- Email desabilitado
- logging em diferentes cenários

- Projeto sem testes

2.2 Criação de Constantes

Ao invés de deixar strings espalhadas, criei a classe `BuildResult` com constantes nomeadas. Isso facilita:

- Mudanças futuras nas mensagens
- Busca por referências na IDE
- Evitar typos

2.3 Extração de Métodos

Dividi o método `run()` em 4 métodos menores:

Método	Responsabilidade
<code>executeTests()</code>	Executa testes e retorna boolean
<code>deployIfTestsPass()</code>	Faz deploy apenas se testes passaram
<code>sendSummaryEmailIfEnabled()</code>	Envia email se configurado
<code>determineEmailMessage()</code>	Determina a mensagem baseada no resultado

2.4 Simplificação de Condicionais

A lógica de email que era:

```
if (testsPassed) {
    if (deploySuccessful) {
        emailer.send("...");
    } else {
        emailer.send("...");
    }
} else {
    emailer.send("...");
}
```

Foi simplificada para:

```
private String determineEmailMessage(boolean testsPassed, boolean deploySuccessful) {
    if (!testsPassed) return EMAIL_TESTS_FAILED;
    if (deploySuccessful) return EMAIL_DEPLOYMENT_COMPLETED;
    return EMAIL_DEPLOYMENT_FAILED;
}
```

3. Dificuldades Encontradas

3.1 Configuração do Ambiente

Problema: O Gradle 6.2.2 não é compatível com Java 21.

Solução: Atualizei o gradle-wrapper.properties para versão 8.5 e ajustei as dependências do build.gradle.

3.2 Testes Não Executavam

Problema: Os testes compilavam mas não eram executados (0 tests found).

Causa: Falta da configuração `useJUnitPlatform()` no build.gradle.

Solução: Adicionei `useJUnitPlatform()` na task de test.

3.3 Compilação Inicial

Problema: Os testes faziam referência a métodos que não existiam no Pipeline original.

Solução: Adicionei overload do método `run(Project)` que aceita `DeploymentEnvironment`

4. Aprendizados

4.1 Importância dos Testes

Trabalhar sem testes em código legado é arriscado. Os testes criados não apenas validaram a refatoração, mas também documentaram o comportamento esperado.

4.2 Extrair Métodos é Fundamental

Métodos pequenos com nomes descritivos tornam o código muito mais legível. Agora posso ler `run()` e entender o fluxo geral sem entrar nos detalhes de cada passo.

4.3 Constantes vs Strings Mágicas

Substituir strings por constantes com nomes significativos melhora muito a manutenibilidade. Se amanhã precisarmos mudar "Deployment failed" para "Build failed", é preciso mexer em um único lugar.

4.4 Simplificação de Condicionais

Condicionais aninhados são difíceis de ler e manter. Extrair a lógica para métodos com nomes descritivos (como `determineEmailMessage`) melhora significativamente a clareza.

5. Conclusão

A refatoração foi bem-sucedida. O código agora é:

- **Mais legível:** Métodos pequenos e nomeados
 - **Mais testável:** 8 testes cobrindo cenários principais
 - **Mais manutenível:** Constantes centralizadas
 - **Mais seguro:** Regressões são protegidas pelos testes
-

6. Comandos Executados

```
# Compilar e testar
./gradlew clean test --no-daemon

# Ver relatório de testes
# build/reports/tests/test/index.html
```